# NotifyHub

Servicio de Notificaciones y Registro Multicanal

Elementos de modelado, arquitectura implementada y metodología

Arquitectura de Software


| | |
|---|---|
| José Raúl Soriano Cazabal | Encargado del proyecto y líder técnico |
| Andreea Margarita Maldonado Morales | Especialista del proyecto |


Repositorio: `github.com/joseraulsoriano/notifyhub` · Tablero: Jira, proyecto SCRUM

Octubre de 2026

# Índice

**1. ¿Qué es NotifyHub? 2**
1.1. Requisitos funcionales . . . 2
1.2. Requisitos no funcionales . . . 2

**2. Metodología: Scrum 2**
2.1. Roles . . . 3
2.2. Sprints y entregables . . . 3
2.3. Artefactos y ceremonias . . . 4
2.4. Definition of Done . . . 4
2.5. Prácticas de ingeniería que sostienen la metodología . . . 4

**3. Arquitectura implementada 4**
3.1. Estilo: hexagonal (puertos y adaptadores) orientada a eventos y *serverless* . . . 4
3.2. Capas y monorepo . . . 5
3.3. Patrones de diseño . . . 5
3.4. Diagrama de contexto . . . 6
3.5. Diagrama de contenedores . . . 6
3.6. Diagrama de despliegue (arquitectura implementada) . . . 7

**4. Diagrama de casos de uso 8**

**5. Diagramas de flujo 9**
5.1. Recepción de una notificación (RF1) . . . 9
5.2. Despacho con reintentos y fallback (RF2, RF3) . . . 10

**6. Diagrama entidad-relación 11**

**7. Diagrama de secuencia 12**

**8. Diagramas complementarios 13**
8.1. Clases y patrones . . . 13
8.2. Máquina de estados de la notificación . . . 13

**9. Diagrama de navegación 14**

**10. Calidad y estado actual 16**

# 1 ¿Qué es NotifyHub?

NotifyHub es un servicio que recibe solicitudes de notificación de otras aplicaciones (un ERP, una tienda en línea, una app móvil) y se encarga de que el mensaje llegue al destinatario por el canal adecuado: correo (SendGrid), SMS (Twilio), notificación push (Firebase Cloud Messaging) o webhook. Si el canal principal falla, reintenta con espera exponencial y, cuando se agota, **conmuta a un canal secundario** (*fallback*). Cada intento y cada cambio de estado queda en una bitácora de auditoría que solo admite inserciones.

El proyecto está construido alrededor de tres patrones GoF que se pueden señalar en el código: **Factory Method** (crear el emisor de cada proveedor), **Strategy** (elegir proveedor por costo, prioridad o disponibilidad) y **Observer** (publicar cada cambio de estado a quien le interese).

## 1.1 Requisitos funcionales

| RF | Descripción |
|---|---|
| RF1 | Recibir solicitudes por REST con API key, validarlas, deduplicarlas por `Idempotency-Key` y encolarlas por prioridad. |
| RF2 | Enrutar cada solicitud al proveedor de su canal mediante factorías configurables (Factory Method). |
| RF3 | Seleccionar proveedor dinámicamente (Strategy) y conmutar a un canal secundario cuando el principal falla. |
| RF4 | Registrar cada intento y transición en una bitácora de auditoría consultable y exportable. |
| RF5 | Notificar los cambios de estado a servicios suscriptores (Observer + outbox transaccional). |

## 1.2 Requisitos no funcionales

- **Confiabilidad:** ninguna solicitud aceptada se pierde; se persiste antes de encolarse.
- **Idempotencia:** reenviar la misma solicitud no duplica el envío (ventana de 24 h).
- **Rendimiento:** meta de 1,000 notificaciones/min con p95 de aceptación menor a 200 ms.
- **Seguridad:** API key por cliente, límite de solicitudes por minuto y enmascarado de datos personales.
- **Costo:** operar en el plan gratuito de Cloudflare (0 USD).
- **Mantenibilidad:** el dominio no depende de ningún framework; cambiar de infraestructura no toca la lógica.

# 2 Metodología: Scrum

El proyecto se gestiona con **Scrum** adaptado a un equipo de dos personas. Se eligió porque el alcance se descompone de forma natural en requisitos funcionales independientes (RF1 a RF5) que se pueden entregar uno por sprint, y porque la entrega de avance y la entrega final funcionan como revisiones de sprint con el profesor.

## 2.1 Roles

| Rol Scrum | Quién y qué hace |
|---|---|
| Product Owner | José Raúl Soriano. Define alcance, prioriza el backlog y acepta historias. |
| Scrum Master / líder técnico | José Raúl Soriano. Coordina, quita bloqueos, cuida la arquitectura y el CI/CD. |
| Equipo de desarrollo | José Raúl Soriano (plataforma, API, infraestructura) y Andreea Maldonado (dominio y patrones GoF, proveedores, auditoría, calidad). |
| Cliente / revisor | El profesor de la materia, en las entregas de avance y final. |

## 2.2 Sprints y entregables

Cinco sprints de dos semanas (del 28 de septiembre al 4 de diciembre de 2026). Cada sprint tiene un objetivo ligado a un requisito funcional; el backlog suma **41 historias y 142 puntos de historia**, repartidos en unos 27–31 puntos por sprint.

| Sprint | Fechas | Objetivo | SP |
|---|---|---|---|
| S0 · Planeación | 28 sep – 9 oct | Requisitos, ADRs, modelos, diagramas, contrato OpenAPI y riesgos | 27 |
| S1 · RF1 | 12 – 23 oct | Monorepo, entorno local, CI, recepción y consulta de notificaciones | 27 |
| S2 · RF2 | 26 oct – 6 nov | Factory Method, adaptadores de proveedores y despachador | 27 |
| S3 · RF3 + RF4 | 9 – 20 nov | Strategy, fallback, circuit breaker y bitácora | 31 |
| S4 · RF5 | 23 nov – 4 dic | Observer, outbox, dashboards, pruebas de carga y entrega final | 30 |

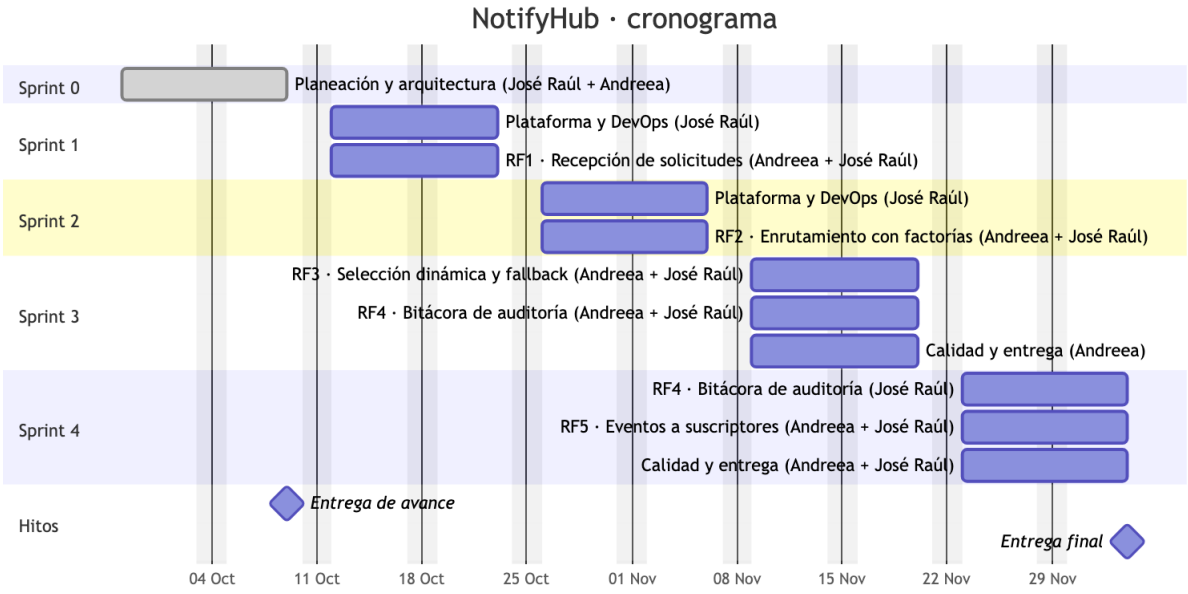



**Figura 1:** Cronograma por sprint y épica.

### 2.3 Artefactos y ceremonias

- **Product backlog** en Jira (proyecto SCRUM), organizado en 8 épicas: planeación, plataforma y DevOps, RF1 a RF5, y calidad y entrega. Cada historia lleva responsable, puntos, prioridad y criterios de aceptación. El backlog también está versionado en el repositorio (`docs/planeacion/backlog.json`).
- **Sprint planning** al inicio de cada sprint: se toman las historias de mayor prioridad del RF del sprint.
- **Seguimiento** en el tablero de Jira; cada commit lleva la clave de la historia, así Jira enlaza el código con la historia automáticamente.
- **Sprint review** en las entregas al profesor; el avance por historia se publica en `docs/avance.md` con evidencia (archivo, prueba o diagrama).
- **Retrospectiva** al cierre: por ejemplo, tras el Sprint 0 se decidió migrar a Cloudflare Workers nativo (ADR-007) para no pagar infraestructura.

### 2.4 Definition of Done

Una historia está terminada cuando: (1) sus criterios de aceptación están cubiertos por al menos una prueba automatizada, (2) el CI está en verde, (3) la otra persona revisó y aprobó el pull request y (4) la documentación está al día (OpenAPI, ADR o README).

### 2.5 Prácticas de ingeniería que sostienen la metodología

- **Ramas cortas por historia** (`SCRUM-22-adaptador-sendgrid`) y pull request con revisión cruzada.
- **Conventional Commits** con la clave de Jira (`feat(api): SCRUM-22 …`), validados con Husky y commitlint.
- **Integración continua** en GitHub Actions: lint, tipos, pruebas, build y una prueba de humo contra Postgres real en cada pull request.
- **Registro de decisiones (ADR):** cada decisión de arquitectura queda escrita con contexto, decisión, consecuencias y alternativas descartadas. Hay 7 ADR; una aceptada no se edita, se sustituye por otra.
- **Gestión de riesgos:** matriz de 10 riesgos con probabilidad, impacto, mitigación y disparador (`docs/riesgos-y-pruebas.md`).

## 3 Arquitectura implementada

### 3.1 Estilo: hexagonal (puertos y adaptadores) orientada a eventos y *serverless*

La arquitectura combina tres ideas:

1. **Hexagonal (puertos y adaptadores).** El centro es el *dominio* (`packages/domain`: entidades, máquina de estados y errores, sin dependencias) y la *aplicación* (`packages/application`: casos de uso, puertos y los patrones de despacho). Todo lo que toca el mundo exterior es un *adaptador* que implementa un puerto: el repositorio en Postgres, la cola, el limitador de solicitudes, los emisores de cada proveedor. ESLint impide que el dominio importe infraestructura.

2. **Asíncrona, orientada a mensajes.** La API solo acepta y encola (responde `202 Accepted`); el envío real lo hace un proceso separado, el despachador, que consume tres colas por prioridad. Así un proveedor lento nunca bloquea a quien llama.
3. **Serverless en el borde.** Todo corre en Cloudflare Workers del plan gratuito: API y despachador son Workers, las colas son Cloudflare Queues, el límite de solicitudes es un Durable Object y la base de datos es Postgres (Neon) a través de Hyperdrive.

La prueba de que la hexagonal funciona: entre el Sprint 0 y el Sprint 1 se cambió NestJS por Hono, BullMQ/Redis por Cloudflare Queues y Prisma Client por `pg`, y los casos de uso y sus pruebas **no cambiaron** (ADR-007).

## 3.2 Capas y monorepo

| Paquete | Responsabilidad |
|---|---|
| `packages/domain` | Entidades, estados y errores. No depende de nada. |
| `packages/application` | Casos de uso, puertos y despacho (Strategy, Factory Method, Observer). |
| `packages/providers` | Emisores por proveedor (Template Method) y sus creadores (Factory Method). |
| `packages/persistence` | Esquema y migraciones Prisma; repositorios con `pg`. |
| `packages/client`, `ui` | SDK de TypeScript y componentes React reutilizables. |
| `apps/api` | Worker Hono: recepción, consulta, proveedores y bitácora. |
| `apps/dispatcher` | Worker consumidor de colas: despacho, reintentos y fallback. |
| `apps/console` | Consola web (Astro + React) para operar el sistema. |

Monorepo con pnpm workspaces y Turborepo, TypeScript estricto. Stack: Hono, Cloudflare Workers/Queues/Durable Objects/Hyperdrive, PostgreSQL con Prisma, Astro + React + shadcn/ui, Vitest, GitHub Actions.

## 3.3 Patrones de diseño

| Patrón | Dónde y para qué |
|---|---|
| Factory Method | `SenderCreator` y `SenderRegistry`: crean el emisor concreto de cada proveedor sin que el despachador conozca las clases concretas. Agregar un proveedor es agregar un creador. |
| Strategy | `CostStrategy`, `PriorityStrategy`, `AvailabilityStrategy`: ordenan los proveedores de un canal; la estrategia se elige según la prioridad de la notificación. |
| Observer | `EventPublisher`: cada cambio de estado e intento se publica; suscriptores como la bitácora, métricas y el outbox reaccionan sin acoplarse al despachador. |
| Template Method / Adapter | `BaseSender` fija el algoritmo (validar, enviar, mapear respuesta) y cada proveedor implementa solo `doSend`. |
| Chain of Responsibility | `FallbackPolicy`: arma la cadena de canales (preferido → secundarios) por la que pasa la notificación. |

## 3.4 Diagrama de contexto

NotifyHub como caja negra: quién lo usa y con quién habla.

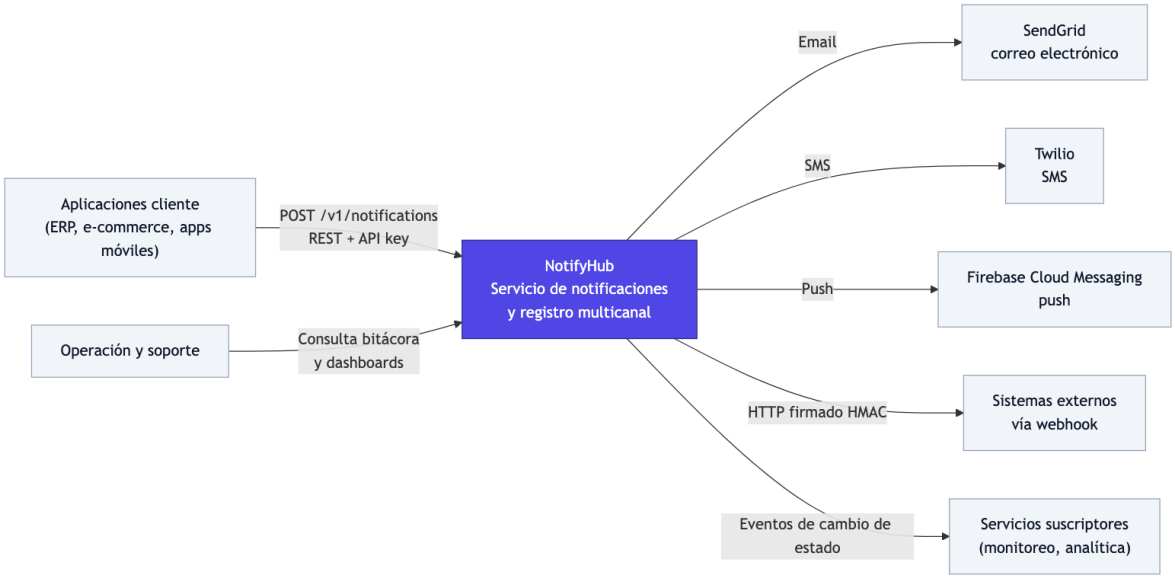



**Figura 2:** Diagrama de contexto (C4 nivel 1).

## 3.5 Diagrama de contenedores

Los procesos y almacenes que forman el sistema. Los clientes llaman a la API, que persiste en PostgreSQL y encola en Queues; el despachador consume la cola y llama a los proveedores; el *event relay* (Sprint 4) leerá el outbox y entregará eventos a los suscriptores. Las líneas punteadas son telemetría (OpenTelemetry).

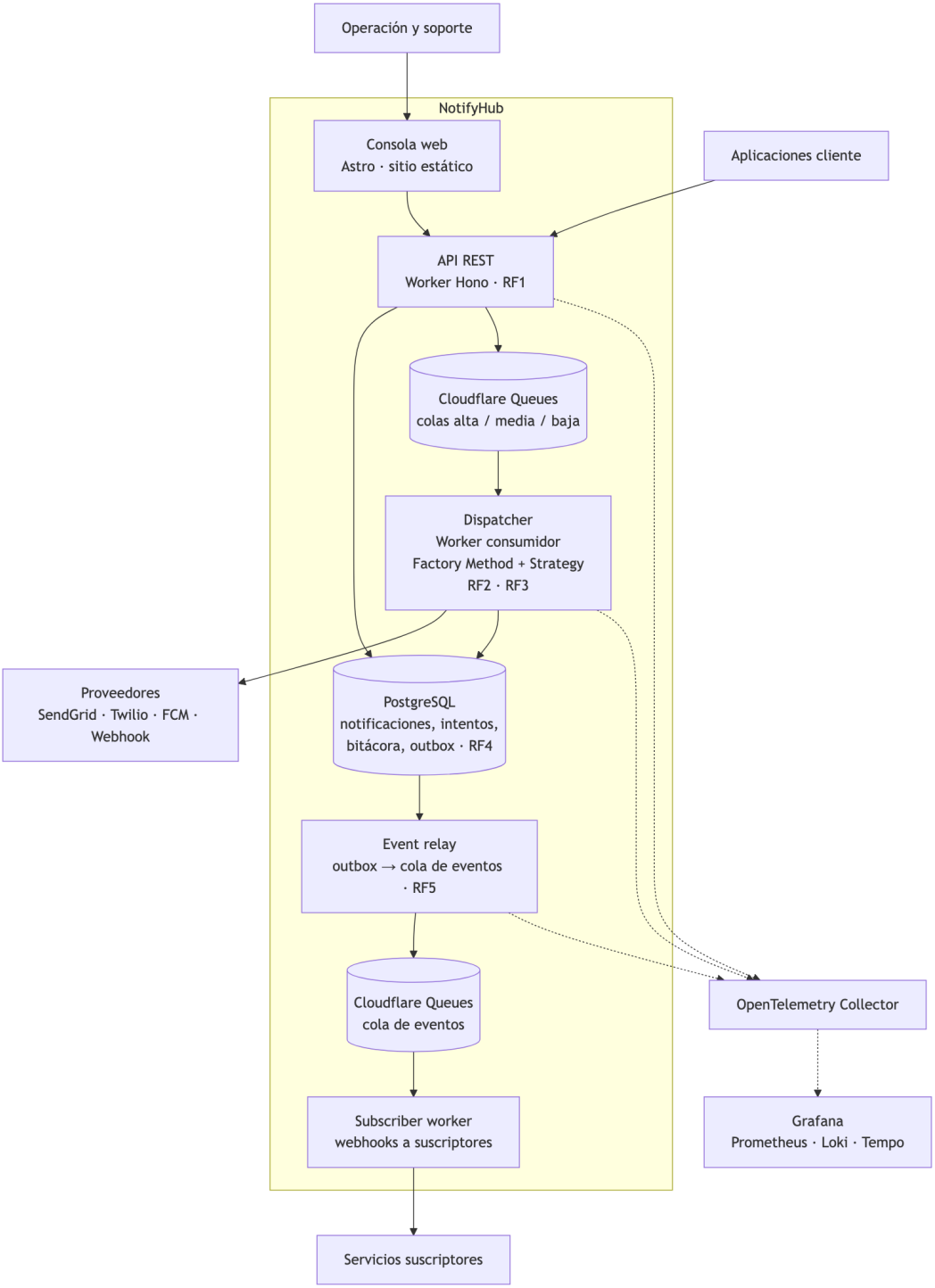



**Figura 3:** Diagrama de contenedores (C4 nivel 2).

## 3.6 Diagrama de despliegue (arquitectura implementada)

Lo que está desplegado hoy en Cloudflare: tres Workers (consola, API y despachador), un Durable Object para el límite de solicitudes, tres colas por prioridad y Hyperdrive como pool de conexiones hacia Neon Postgres. El código vive en GitHub, donde Actions corre el CI en cada pull request; el despliegue se hace con `wrangler deploy`.

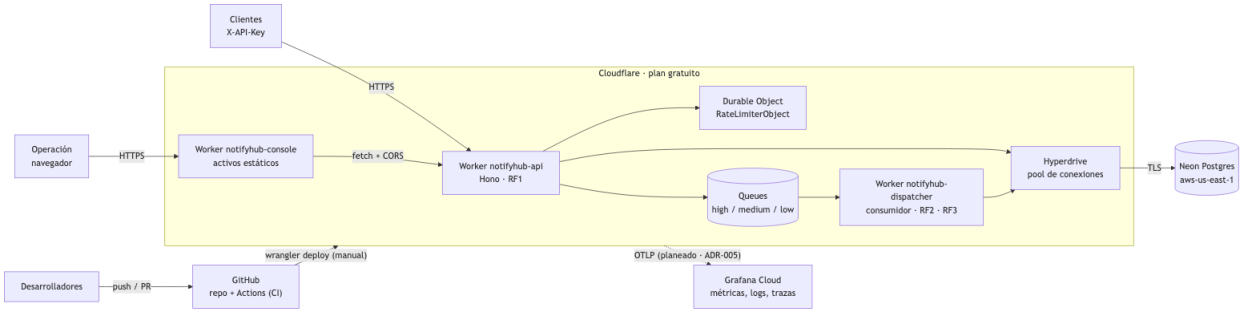



**Figura 4:** Despliegue en Cloudflare (plan gratuito).

**Funcionamiento de punta a punta.**

1. Una aplicación cliente hace `POST /v1/notifications` con su API key y, opcionalmente, una `Idempotency-Key`.
2. El Worker de la API valida la llave, consulta al Durable Object si el cliente está dentro de su límite, valida el cuerpo y revisa la idempotencia.
3. Guarda la notificación en Postgres con estado `RECEIVED`, la encola en la cola de su prioridad, la pasa a `QUEUED` y responde `202` con el `id`.
4. El Worker despachador toma el mensaje, la `FallbackPolicy` arma la cadena de canales, la Strategy ordena los proveedores y el `SenderRegistry` crea el emisor.
5. Si el proveedor falla de forma transitoria se reintenta hasta 3 veces con backoff exponencial; si se agota, pasa al siguiente proveedor o canal. Cada intento se guarda y se publica como evento.
6. El cliente o el operador consultan el estado con `GET /v1/notifications/{id}` o en la consola, que dibuja la ruta del mensaje: cada canal es una línea, cada intento una estación y cada fallback un transbordo.

**Qué es real y qué es simulado.** Es real la recepción con idempotencia y límite de solicitudes, las colas por prioridad, la máquina de estados, los reintentos, el fallback, la bitácora en Postgres, la API, el SDK y la consola. Los proveedores están **simulados**: su comportamiento depende de su salud configurada (`UP`, `DEGRADED`, `DOWN`), lo que permite demostrar el fallback sin credenciales. Outbox y suscripciones (RF5) y circuit breaker están planeados para S3–S4.

## 4 Diagrama de casos de uso

Actores humanos a la izquierda (aplicación cliente, operador, administrador) y sistemas externos a la derecha (cola/despachador, proveedores y suscriptores). Las relaciones «include» son pasos obligatorios (toda llamada se autentica; todo despacho reintenta y publica eventos); las «extend» son opcionales (la deduplicación solo ocurre si llega `Idempotency-Key`; el fallback solo si se agota el canal; exportar CSV es opcional al consultar la bitácora).

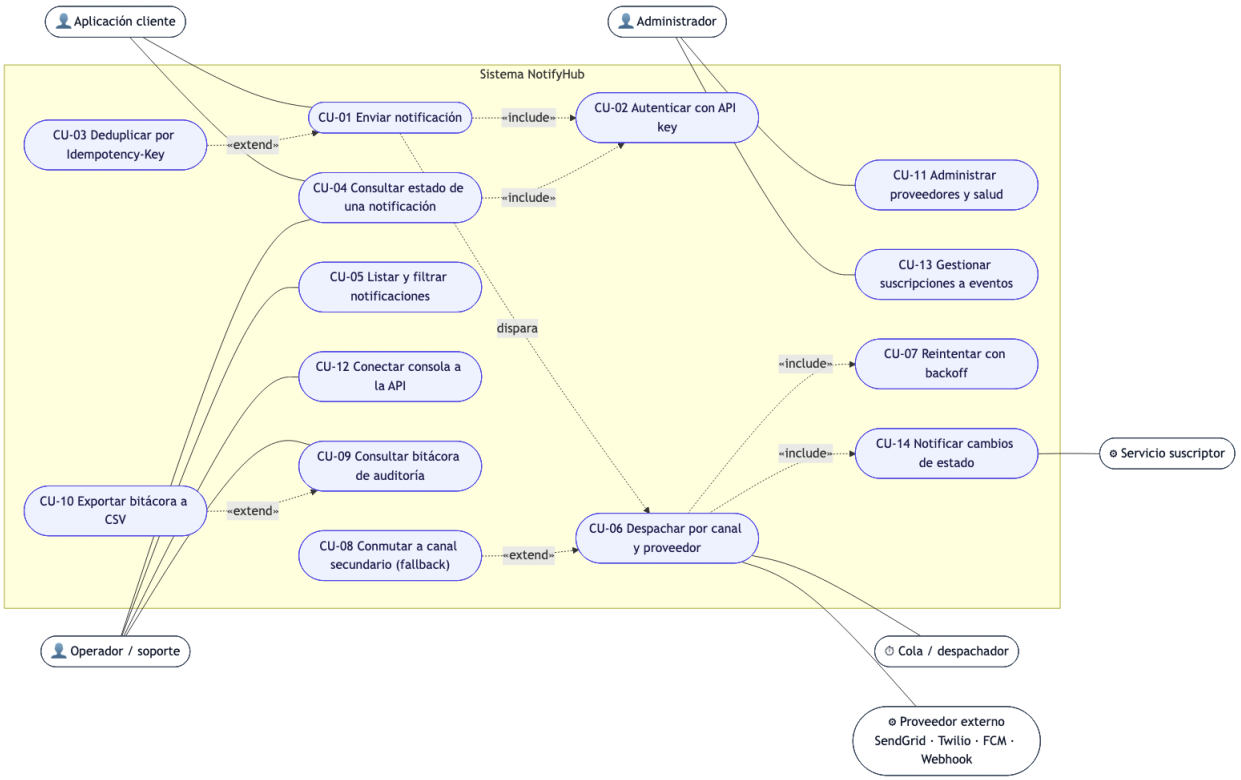



**Figura 5:** Diagrama de casos de uso.

| CU | Actor | Descripción breve |
|---|---|---|
| CU-01 | App cliente | Envía una notificación con remitente, destinatario, mensaje, prioridad y canal preferido. |
| CU-04 | App cliente, operador | Consulta estado, intentos e historial de una notificación. |
| CU-05 | Operador | Lista notificaciones filtrando por estado, canal y prioridad, con paginación. |
| CU-06 | Despachador | Entrega la notificación por el mejor proveedor del canal. |
| CU-08 | Despachador | Conmuta al canal secundario cuando el principal se agota. |
| CU-09/10 | Operador | Consulta la bitácora de auditoría y la exporta a CSV. |
| CU-11 | Administrador | Habilita proveedores y cambia su salud, lo que afecta el ruteo en vivo. |
| CU-13/14 | Administrador, suscriptor | Gestiona suscripciones y recibe eventos de cambio de estado (S4). |

# 5 Diagramas de flujo

## 5.1 Recepción de una notificación (RF1)

Las validaciones van de la más barata a la más cara: llave, límite, esquema e idempotencia. Lo importante del diseño es que la notificación se **persiste antes de encolarse**: si la cola falla, queda en `RECEIVED` y el despachador la recupera en un barrido, así nunca se pierde.

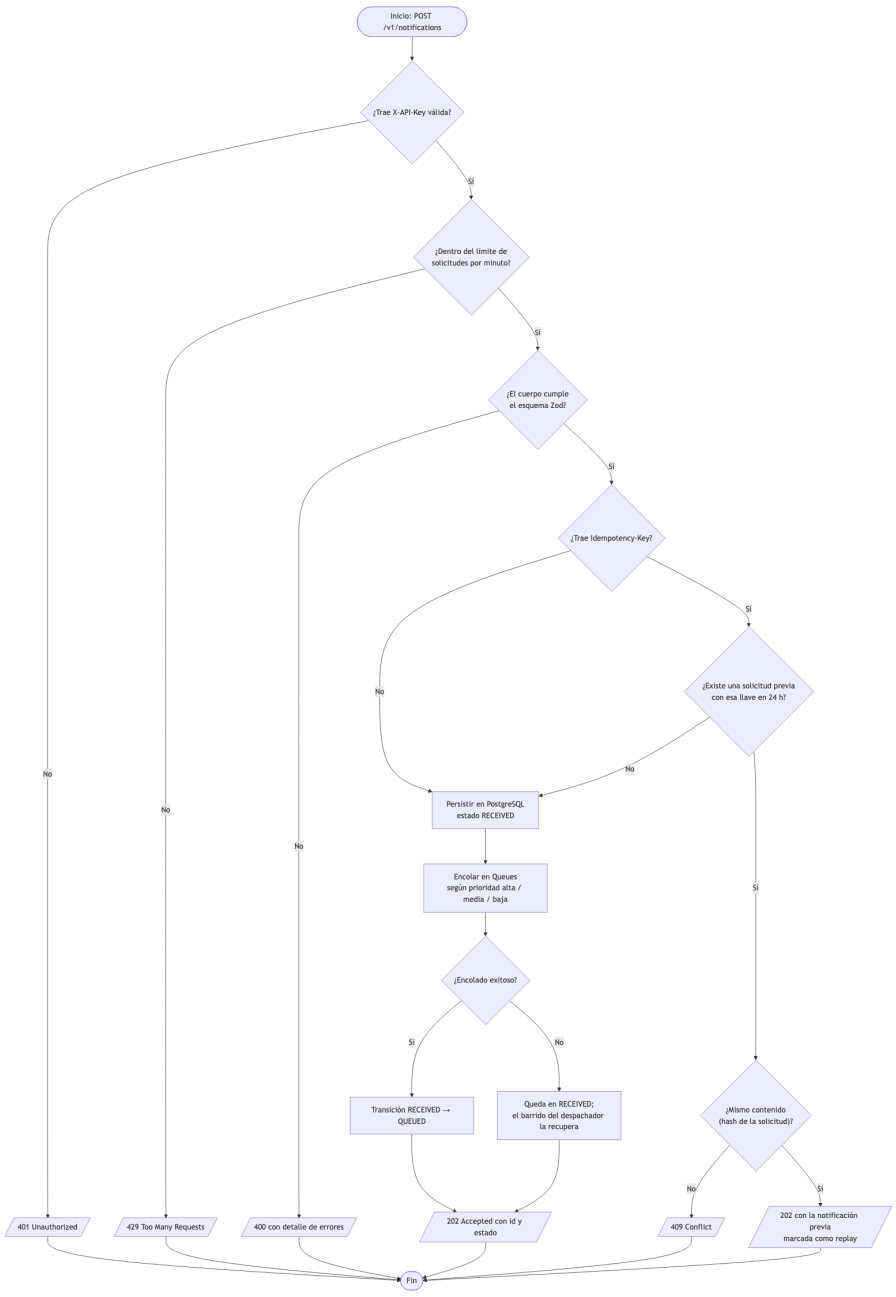



**Figura 6:** Flujo de recepción en la API.

## 5.2 Despacho con reintentos y fallback (RF2, RF3)

Es el algoritmo del `DispatchService`. Por cada canal de la cadena, la Strategy ordena los proveedores y la factoría crea el emisor. Un error transitorio provoca reintentos con backoff; un error permanente o los reintentos agotados pasan al siguiente proveedor y, después, al siguiente canal. Como Queues entrega *al menos una vez*, lo primero que hace es verificar que la notificación siga en `QUEUED`.

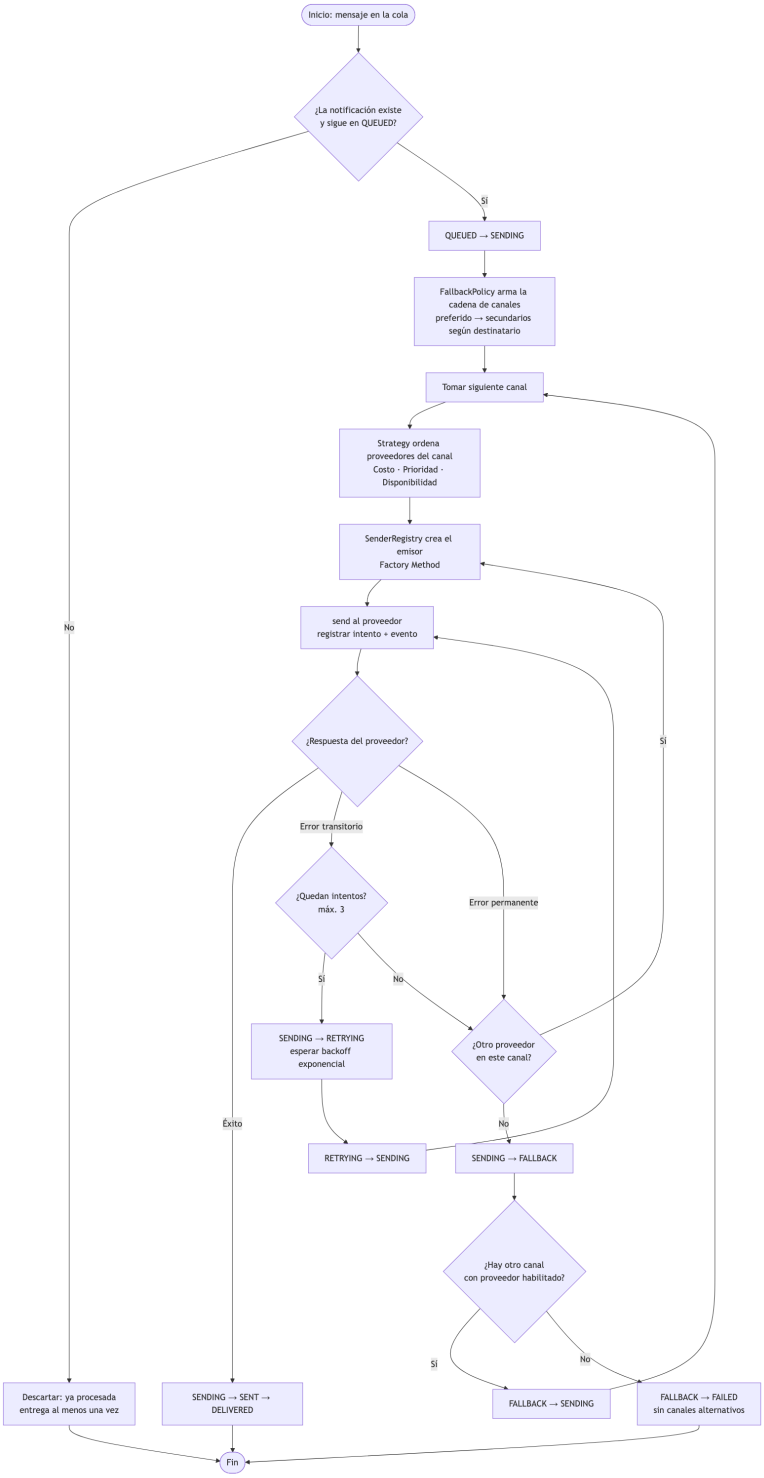



**Figura 7:** Flujo de despacho en el Worker despachador.

## 6 Diagrama entidad-relación

`NOTIFICATION` es la entidad central: pertenece a un `CLIENT_APP`, tiene muchos `DELIVERY_ATTEMPT` (cada uno hecho por un `PROVIDER`) y deja registros en `AUDIT_LOG`. `ROUTING_RULE` define el orden de proveedores y el canal de fallback por prioridad. Para RF5, `OUTBOX_EVENT` guarda los eventos en la misma transacción que el cambio de estado, y `SUBSCRIPTION_DELIVERY` registra la entrega de cada evento a cada suscripción. La llave `(client_app_id, idempotency_key)` es única para garantizar la deduplicación.

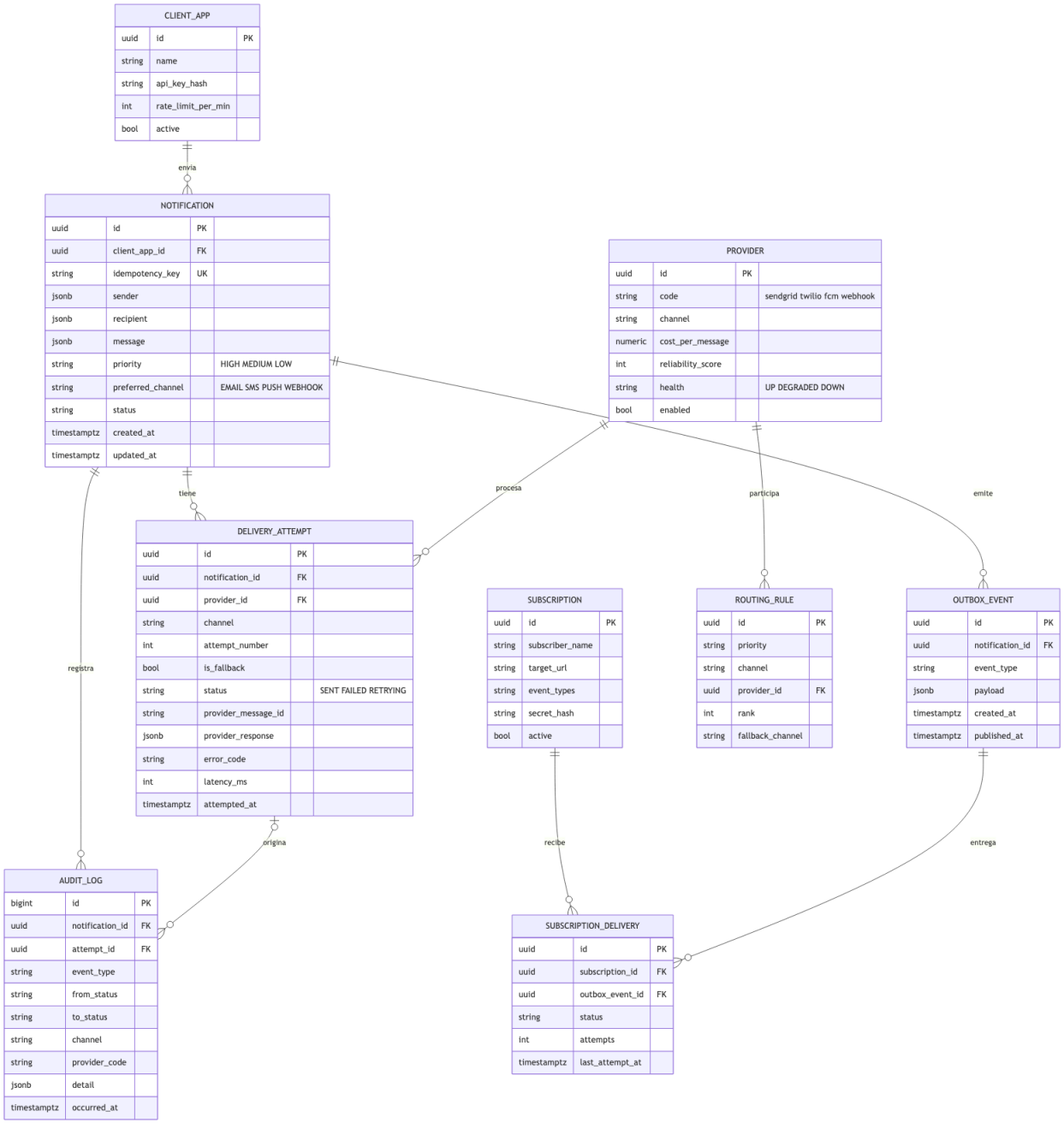



**Figura 8:** Modelo entidad-relación.

## 7 Diagrama de secuencia

Envío de una notificación con los tres desenlaces posibles: éxito directo, error transitorio con reintento y fallback a un proveedor secundario. Muestra cómo colaboran los patrones: el `DispatchService` pide el orden a la Strategy, el emisor a la factoría y publica cada cambio al `EventPublisher` (Observer).

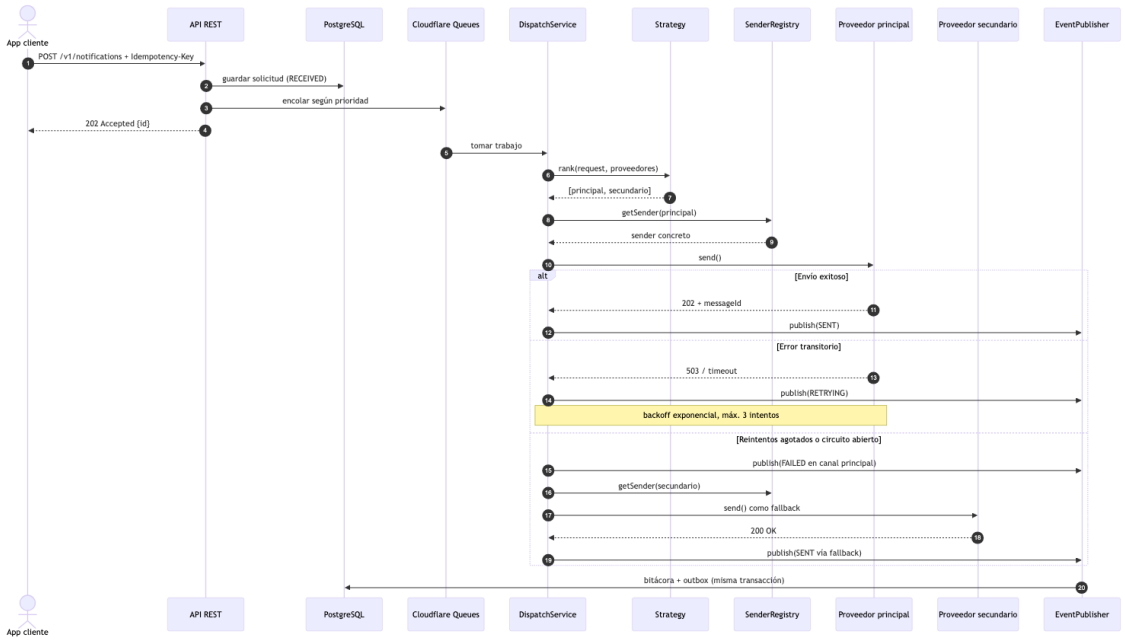



**Figura 9:** Secuencia de envío con fallback.

# 8 Diagramas complementarios

## 8.1 Clases y patrones

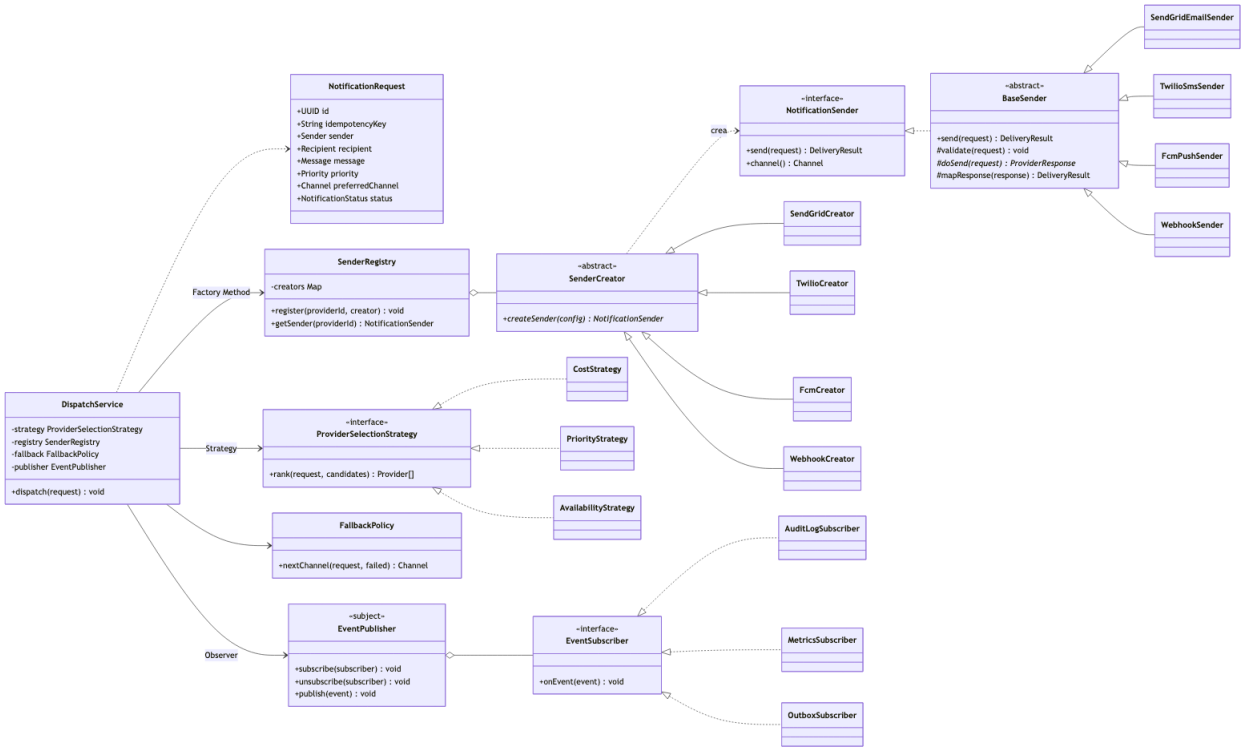



**Figura 10:** Diagrama de clases con Factory Method, Strategy y Observer.

## 8.2 Máquina de estados de la notificación

Cada transición solo procede desde el estado esperado; eso hace idempotente al despachador.

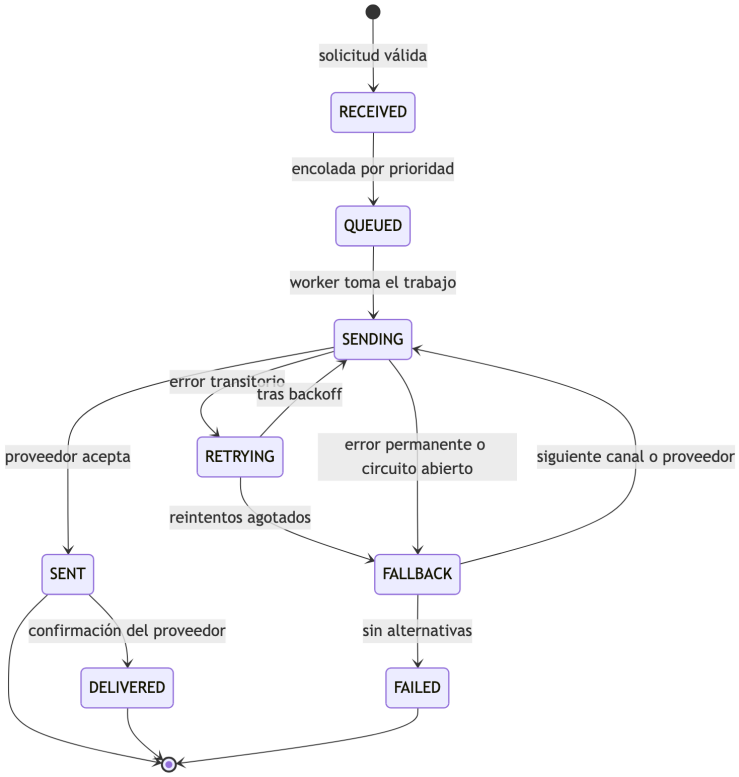



**Figura 11:** Estados de una notificación.

## 9 Diagrama de navegación

La consola web (`apps/console`) tiene seis pantallas agrupadas en la barra lateral: *Operación* (Resumen, Notificaciones, Nueva solicitud), *Configuración* (Proveedores y ruteo, Suscripciones) y *Control* (Bitácora). Sin conexión guardada arranca en modo demo con un cliente simulado; desde el diálogo *Conectar* se pasa a la API real. El panel de detalle muestra la ruta del mensaje y enlaza con la bitácora filtrada por esa notificación.

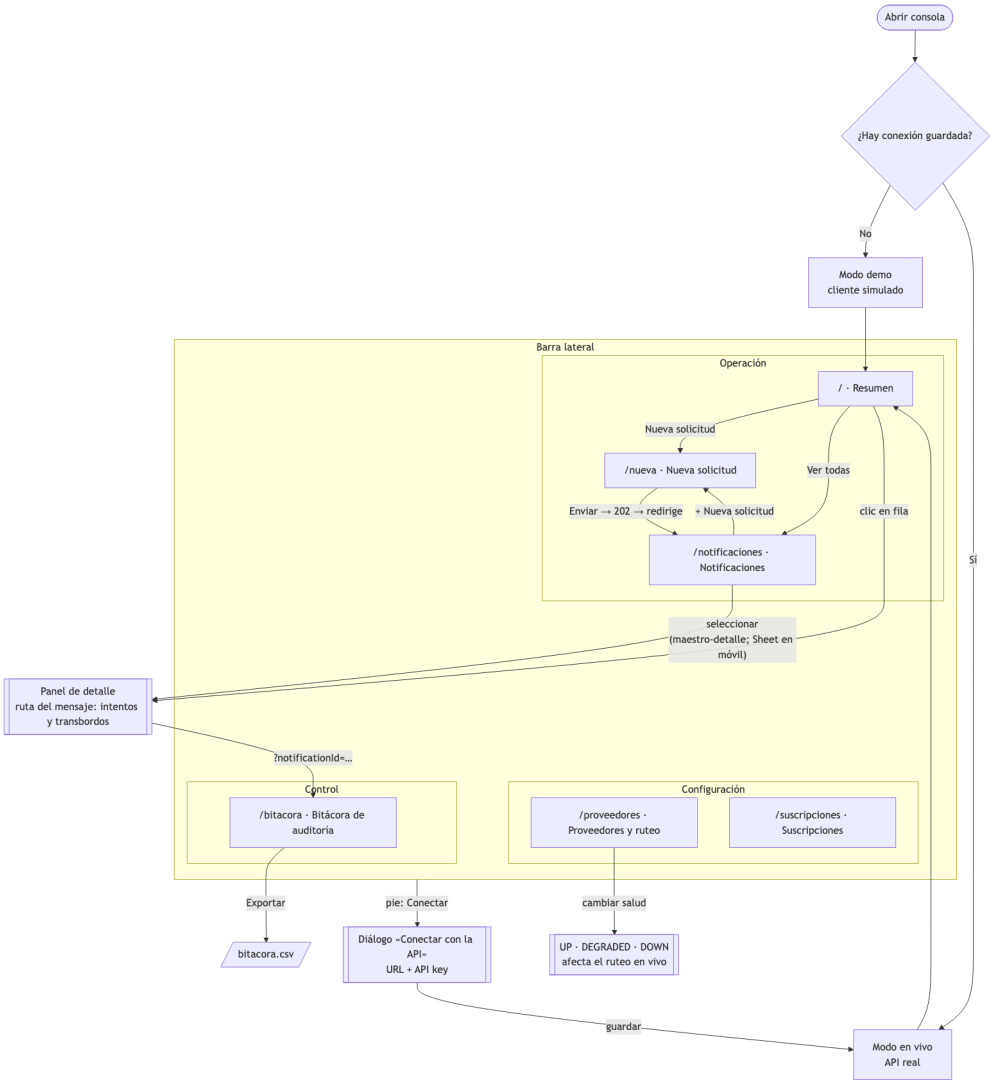



**Figura 12:** Diagrama de navegación de la consola.

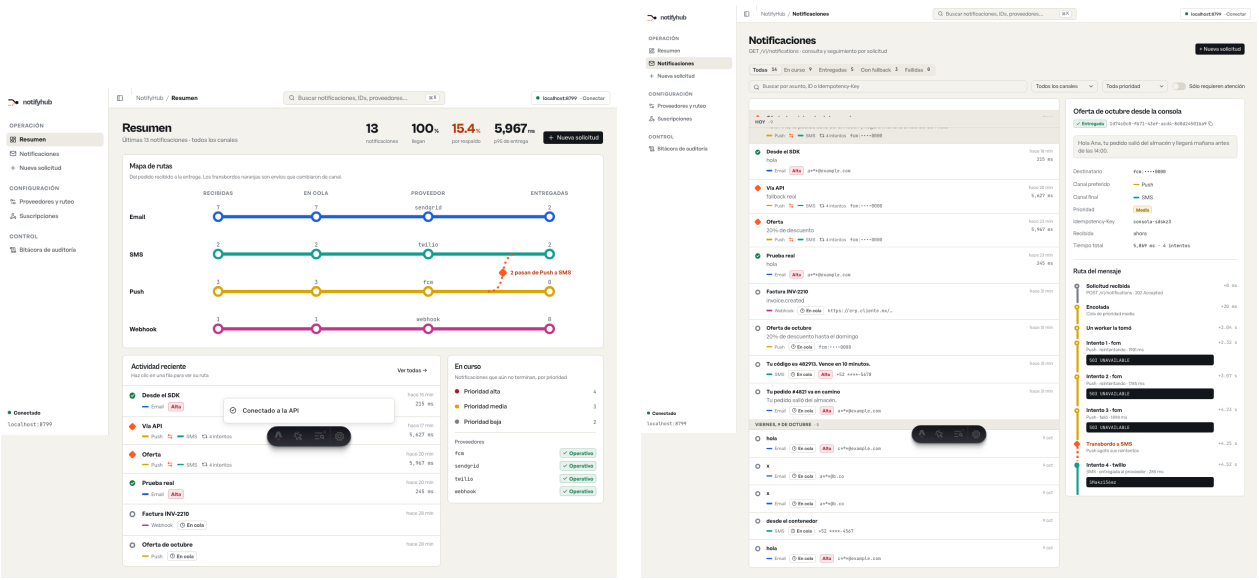



**Figura 13:** Pantallas Resumen y detalle de una notificación con fallback (tres intentos en Push y transbordo a SMS).

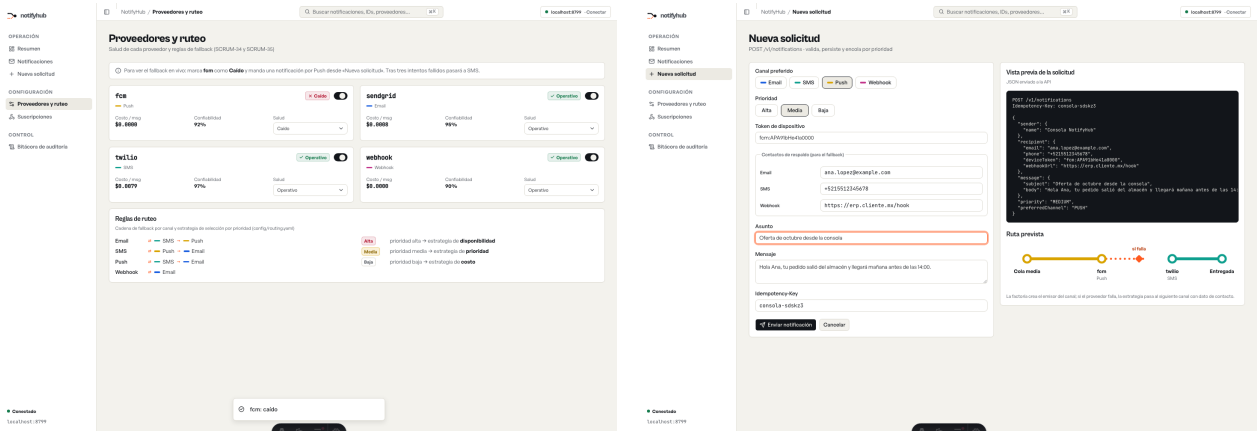



**Figura 14:** Pantallas Proveedores y ruteo, y Nueva solicitud.

## 10 Calidad y estado actual

- **148 pruebas automatizadas en verde** (Vitest) en dominio, aplicación, proveedores, API, SDK, UI y consola, más una prueba de humo contra Postgres real en CI.
- Pirámide de pruebas: unitarias (máquina de estados, casos de uso, estrategias, fallback), de componente (cada código de respuesta del contrato OpenAPI), integración, humo y carga con k6 (S4).
- Contrato de la API en OpenAPI 3.1 (`docs/api/openapi.yaml`), validado con Redocly.
- Desplegado y verificado en Cloudflare: API, despachador y consola.
- Pendiente: adaptadores reales de proveedores, circuit breaker, outbox y suscripciones (RF5), dashboards y prueba de carga.